Full-Stack Quest

Coordination développement Front et Back

Le contrat d'abord

Contract-first, OpenAPI, mocks

Rejoindre la classe



localhost:5173/rejoindre

Date

Mardi 27 octobre 2026

Horaires · salle

9 h – 13 h · S102

Chapitre

02 / 13

Durée

4 h

Jeu du jour

Contract Tetris N5-8

Mastère 2 Expert en développement Full Stack · A2627_0195

Le déroulé du jour

Mardi 27 octobre 2026 · 9 h – 13 h

9 h 00		**Contract Tetris N5-8** 15 min Seul jeu de la séance : codes de statut, erreurs au format commun, pagination, champs facultatifs.
9 h 15		Théorie 1 · Le contrat d'abord 40 min
9 h 55		Rédiger le contrat de l'équipe 55 min
10 h 50		Pause 10 min
11 h 00		Théorie 2 · Le contrat d'intégration v1 10 min
11 h 10		Mocks des deux côtés 1 h 30
12 h 40		Revue croisée des contrats 20 min

Contract Tetris

Niveaux 5 à 8 · Solo · manches de 3 min

01 Ouvre **Contract Tetris**, niveaux 5 à 8.

02 Nouveautés : **codes de statut**, erreurs au format commun, **pagination**, champs facultatifs.

03 Lis l’écran de fin : chaque erreur cite la règle et le champ fautif.

Temps de jeu

15:00

▶ Cliquer pour lancer

Compétence

Lire un contrat REST d'un coup d'œil

Où jouer

localhost:5173 → hub → Contract Tetris

Le contrat d’abord

Théorie 1 · 9 h 15 · 40 min

01 Contract-first ou code-first

02 Anatomie d'OpenAPI 3.1

03 Erreurs, exemples, lint

04 Mocks avec Prism et MSW

Deux façons d’obtenir un contrat

	Code-first	Contract-first
Point de départ	Le code du back, annoté	Un fichier OpenAPI relu par les deux côtés
La spec est…	générée, donc toujours « vraie » pour le back	écrite, donc vraie pour **les deux**
Le front peut commencer…	quand le back a codé l’endpoint	dès que la PR du contrat est fusionnée
Un changement passe par…	une PR de code, la spec suit	une PR de contrat, le code suit
Risque principal	l’API épouse le modèle interne du back	le code dérive du contrat sans test

Dès que deux équipes consomment l’API, le contrat passe en premier.

Code-first convient à une équipe seule qui possède les deux côtés. Entre équipes, une spec générée décrit ce que le back **a fait**, pas ce dont le front **a besoin**.

Le contrat d’abord, concrètement



- La **PR sur le contrat** est la réunion : les désaccords se règlent en commentaires, ligne par ligne.
- Une fois fusionné, le contrat débloque **les deux côtés en même temps**.
- Plus tard (19 nov.), des **tests de contrat** vérifient que le code tient la promesse.

Anatomie d’un fichier OpenAPI 3.1

```yaml
openapi: 3.1.0
info: { title: Cache Racer, version: 0.3.0 }
servers:
  - url: http://localhost:4000
paths:
  /api/levels/{levelId}:
    get: { … }
components:
  schemas: { … }           # formes réutilisables (JSON Schema)
  responses: { … }         # réponses réutilisables (erreurs)
  parameters: { … }        # paramètres réutilisables (pagination)
  securitySchemes: { … }   # comment on s'authentifie
security:
  - session: []
webhooks: { … }            # ce que VOUS appelez chez le client (3.1)
```

- `info.version` : la version **du contrat**, pas du code
- `paths` : les opérations, une par verbe
- `components` : tout ce qui se réutilise par `$ref`
- `security` au niveau racine : la règle par défaut
- `webhooks` : nouveauté de la 3.1

Ce qui change entre 3.0 et 3.1

```
 Player:
   type: object
   properties:
     avatarUrl:
-      type: string
-      nullable: true
-      example: https://cdn.example/neo.png
+      type: [string, "null"]
+      format: uri
+      examples: [https://cdn.example/neo.png]
     level:
       type: integer
-      minimum: 0
-      exclusiveMinimum: true
+      exclusiveMinimum: 0
```

- 3.1 adopte **JSON Schema 2020-12** tel quel
- `nullable` disparaît : `null` est un type
- `examples` est un tableau
- `exclusiveMinimum` porte la valeur
- Conséquence : les validateurs JSON Schema (Ajv, pydantic) lisent vos schémas **sans traduction**

Une opération complète

```yaml
/api/levels/{levelId}/runs:
  post:
    operationId: createRun
    parameters:
      - name: levelId
        in: path
        required: true
        schema: { type: string, pattern: "^[a-z0-9-]+$" }
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: "#/components/schemas/RunInput" }
    responses:
      "201":
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Run" }
      "404": { $ref: "#/components/responses/NotFound" }
      "422": { $ref: "#/components/responses/ValidationError" }
```

- `operationId` : le nom de la fonction **générée** côté client
- Le paramètre de chemin est **requis** et contraint
- Le corps est requis ; sa forme est un `$ref`
- Chaque code de statut possible est **déclaré**

Des schémas qui tranchent

```yaml
RunInput:
  type: object
  required: [score, durationMs]
  additionalProperties: false
  properties:
    score: { type: integer, minimum: 0, maximum: 10000 }
    durationMs: { type: integer, minimum: 0 }
    hintsUsed: { type: integer, minimum: 0, default: 0 }
Result:
  oneOf:
    - $ref: "#/components/schemas/Win"
    - $ref: "#/components/schemas/Loss"
  discriminator:
    propertyName: outcome
```

- `required` : ce qui manque est une **erreur**
- `additionalProperties: false` sur les **requêtes** : un champ inconnu est refusé (422)
- Bornes explicites : le back n'a plus à les deviner
- `oneOf` + `discriminator` : un champ dit quelle variante lire

Un seul format d’erreur, partout

Le format du contrat FSQ

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "score : entier attendu"
  }
}
```

RFC 9457 · Problem Details

```
{
  "type": "/errors/validation",
  "title": "Corps invalide",
  "status": 422,
  "detail": "score doit être un entier"
}
```

- Les deux se défendent. Ce qui ne se défend pas : **en mélanger deux**, ou renvoyer du HTML.
- Le `code` est **stable** et sert aux programmes ; le `message` peut changer et sert aux humains.

Les exemples font partie du contrat

```yaml
responses:
  "200":
    content:
      application/json:
        schema: { $ref: "#/components/schemas/Level" }
        examples:
          premier:
            summary: Niveau 1, sans cache
            value: { id: cache-froid, par: 1200, locked: false }
          verrouille:
            summary: Niveau pas encore débloqué
            value: { id: cdn-mondial, par: 400, locked: true }
```

- Les **mocks** renvoient ces exemples
- La **documentation** les affiche
- Un exemple par **cas qui change l'écran** : vide, verrouillé, erreur
- Un exemple invalide est un bug : le lint le détecte

Le lint : la relecture que personne n’oublie

```yaml
# .spectral.yaml
extends: ["spectral:oas"]
rules:
  operation-operationId: error
  oas3-valid-media-example: error
  fsq-erreurs-communes:
    description: Toute opération déclare au moins une erreur 4xx
    given: "$.paths[*][get,post,put,patch,delete].responses"
    then:
      function: schema
      functionOptions:
        schema: { patternProperties: { "^4": {} }, minProperties: 2 }
```

```bash
npx @stoplight/spectral-cli lint contracts/openapi.yaml \
  --fail-severity=warn
```

- Les règles `spectral:oas` couvrent la validité OpenAPI
- On **monte** certaines règles en erreur
- On écrit ses **propres règles** : conventions de l’équipe
- En CI, un avertissement bloque la PR

Deux mocks, deux usages

Front · MSW

- Le front **simule le back de l'équipe**
- Interception **dans le navigateur** (Service Worker)
- Les requêtes réelles partent : l'onglet Réseau les montre
- Idéal pour les **cas d'écran** : vide, lent, erreur 422

Les deux lisent le même contrat

Back · Prism

- Le back **simule le hub** à partir de `integration-v1.yaml`
- Un **vrai serveur HTTP**, sans écrire une ligne
- Valide les requêtes contre le contrat
- Idéal pour développer **avant d'avoir une gameKey**

Prism : le hub en local, en une commande

```
npx @stoplight/prism-cli mock contracts/integration-v1.yaml --port 4010

# un joueur, tiré de l'exemple du contrat
curl -H "Authorization: Bearer test" localhost:4010/api/v1/me

# forcer un cas d'erreur
curl -H "Prefer: code=401" localhost:4010/api/v1/me

# une requête invalide est rejetée par Prism lui-même
curl -X POST localhost:4010/api/v1/scores \
  -H "Authorization: Bearer test" -H "X-Game-Key: dev" \
  -H "Content-Type: application/json" \
  -d '{ "gameId": "cache-racer", "level": "3", "score": 10 }'
```

- Réponses tirées des **exemples** du contrat
- `Prefer: code=…` et `Prefer: example=…` choisissent le cas
- `--dynamic` génère des données à partir des schémas
- Le dernier appel renvoie **422** : `level` est une chaîne

MSW : le back, simulé dans le navigateur

```ts
// src/mocks/handlers.ts
import { http, HttpResponse, delay } from "msw";
// types générés depuis contracts/openapi.yaml
import type { components } from "../api/schema";

type Level = components["schemas"]["Level"];

const invalid = {
  error: { code: "VALIDATION_ERROR", message: "score manquant" },
};

export const handlers = [
  http.get("/api/levels/:levelId", () => {
    const level: Level = { id: "cache-froid", par: 1200, locked: false };
    return HttpResponse.json(level);
  }),
  http.post("/api/levels/:levelId/runs", async () => {
    await delay(800);
    return HttpResponse.json(invalid, { status: 422 });
  }),
];
```

- Syntaxe MSW 2 : `http.*` et `HttpResponse`
- Le type `Level` vient **du contrat** : un mock qui dérive ne compile plus
- `delay()` pour voir les états de chargement
- Démarrage conditionnel : `worker.start()` si `VITE_MOCKS=1`

Des types générés, pas recopiés

```
npx openapi-typescript contracts/openapi.yaml \
  -o apps/web/src/api/schema.d.ts
```

```
import createClient from "openapi-fetch";
import type { paths } from "./schema";

const api = createClient<paths>({ baseUrl: "/" });

const { data, error } = await api.POST("/api/levels/{levelId}/runs", {
  params: { path: { levelId: "cache-froid" } },
  body: { score: 1850, durationMs: 94000 },
});
```

- Le chemin, les paramètres et le corps sont **vérifiés à la compilation**
- `data` et `error` sont typés selon les réponses déclarées
- Le contrat change, la génération change, **le front casse au build**, pas en démo

Projet · 9 h 55 – 10 h 50

Rédiger le contrat de l’équipe

01 Listez les **écrans** du niveau 1 et ce que chacun doit afficher.

02 Déduisez les **opérations** ; nommez-les (`operationId`).

03 Écrivez les **schémas** : requis, bornes, `additionalProperties` .

04 Déclarez les **erreurs** au format choisi et un **exemple** par cas d’écran.

05 Ouvrez la PR : le front relit le back, le back relit le front.

Livrable de fin de séance

- ☐ `contracts/openapi.yaml` : au moins les opérations du niveau 1
- ☐ Chaque opération avec ses erreurs et un exemple
- ☐ Spectral sans erreur

Rendu : PR « contrat v0 » ouverte, relue par l'autre sous-groupe

Le contrat d’intégration v1

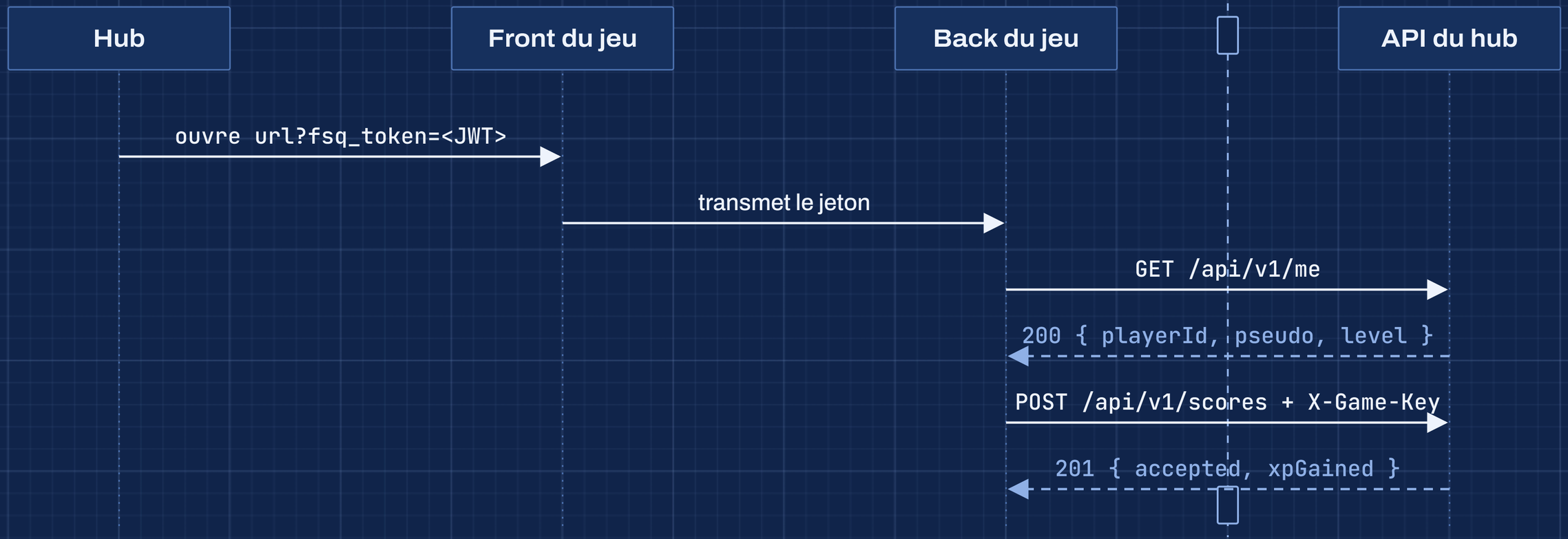
Théorie 2 · 11 h 00 · 10 min

01 Le parcours d'une partie

02 Les quatre points de contact

03 Le demo-game

Le parcours d’une partie



Quatre points de contact, quatre règles

Point	Contrat	La règle qui casse le plus souvent
Lancement	`<url>?fsq_token=<JWT>`	Le front envoie le jeton **à son back**, jamais au hub
Manifeste	`GET <url>/fsq-manifest.json`	`id` en minuscules et tirets, `contractVersion: "1.0"`
Joueur	`GET /api/v1/me` (Bearer)	Appelé **par le back**, à chaque nouvelle partie
Score	`POST /api/v1/scores` (Bearer + `X-Game-Key`)	`level` est un **entier** ; aucun champ en plus (422)

Toutes les erreurs : `{ "error": { "code", "message" } }`.
Débit : 60 requêtes par minute.

Le demo-game : la référence à lire

Dans le dépôt de la plateforme : `examples/demo-game`. Un jeu minimal qui respecte le contrat v1 de bout en bout.

- Un **front** qui lit `fsq_token` dans l'URL et le passe à son back
- Un **back** qui appelle `/me`, puis `/scores` avec la clé lue dans l'environnement
- Un **manifeste** servi à la racine
- Le cas d'erreur : jeton expiré, clé invalide

Lisez-le, ne le copiez pas

Votre jeu a son propre contrat interne et sa propre stack. Le demo-game montre **où passe la jointure**, pas comment écrire votre jeu.

Cherchez-y : où est lue la clé, qui appelle le hub, ce qui se passe sur un 401.

Projet · 11 h 10 – 12 h 40

Mocks des deux côtés

Front

01 Générez les types : `openapi-typescript`.

02 Écrivez les handlers MSW des écrans du niveau 1.

03 Couvrez **un cas d'erreur** par écran.

Back

01 Lancez Prism sur `integration-v1.yaml`.

02 Appelez `/me`, puis `/scores` avec un corps **valide**.

03 Servez le manifeste ; validez-le contre le schéma `Manifest`.

Livrable de fin de séance

- ☐ Front branché sur MSW, types générés du contrat
- ☐ Back qui appelle le hub simulé par Prism (/me puis /scores)
- ☐ `/fsq-manifest.json` servi en local

Rendu : démo de 2 min à l'intervenant avant 12 h 40

Projet · 12 h 40 – 13 h 00

Revue croisée des contrats

Équipe 1 relit 2, 2 relit 3, 3 relit 1. **La grille en 10 points** :

01 Chaque opération a un `operationId`

02 Chaque champ requis l'est vraiment

03 Types et formats précis (dates, URL)

04 Nullabilité explicite

05 Bornes sur les nombres et les chaînes

06 Erreurs déclarées, format unique

07 Un exemple par cas d'écran

08 Pagination décrite si liste

09 Sécurité déclarée par opération

10 Rien qui expose le modèle interne du back

Livrable de fin de séance

- ☐ Deux remarques bloquantes au maximum
- ☐ Le reste en suggestions, dans la PR de l'autre équipe

Rendu : commentaires déposés dans la PR relue

Ce qu'on retient

Vu aujourd'hui

- **Contract-first** dès que deux équipes partagent une API ; la PR de contrat est la réunion
- **OpenAPI 3.1** = JSON Schema 2020-12 : `type: [string, "null"]`, `examples`
- Strict en entrée (`additionalProperties: false`), extensible en sortie
- Un **format d'erreur** unique, des codes stables
- **Mocks** : MSW côté front, Prism côté back, tous deux nourris par le contrat
- **Types générés** : le mock et le client ne peuvent plus mentir

À rendre

- [ ] `contracts/openapi.yaml` de l'équipe, valide (Spectral sans erreur), avec exemples.
- [ ] Front qui tourne sur MSW ; back qui appelle le hub simulé par Prism.
- [ ] `/fsq-manifest.json` servi en local et conforme au schéma `Manifest`.

Prochaine séance

Vendredi 30 octobre 2026

Négocier l'interface

Git, monorepo, premier CI



API Diplomacy

Niveaux 1 à 3

Négocier une interface à deux